

ΣΧΟΛΗ ΜΗΧΑΝΙΚΩΝ

ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ ΠΛΗΡΟΦΟΡΙΚΗΣ ΚΑΙ ΗΛΕΚΤΡΟΝΙΚΩΝ ΣΥΣΤΗΜΑΤΩΝ

ΔΙΠΛΩΜΑΤΙΚΗ ΕΡΓΑΣΙΑ
«ΘΕΜΑ»

«Εικόνα»

Τ... φοιτητ.....
.....
Αρ. Μητρώου: .....

Επιβλέπων
Ονοματεπώνυμο .....
Βαθμίδα .....

30 Οκτωβρίου 2026

Τίτλος Δ.Ε. ....
Κωδικός Δ.Ε. ....
Ονοματεπώνυμο φοιτητή/ών ....
Ονοματεπώνυμο εισηγητή ....
Ημερομηνία ανάληψης Δ.Ε. ....
Ημερομηνία περάτωσης Δ.Ε. ....

Βεβαιώνω ότι είμαι ο συγγραφέας αυτής της εργασίας και ότι κάθε βοήθεια την οποία είχα για την προετοιμασία της είναι πλήρως αναγνωρισμένη και αναφέρεται στην εργασία. Επίσης, έχω καταγράψει τις όποιες πηγές από τις οποίες έκανα χρήση δεδομένων, ιδεών, εικόνων και κειμένου, είτε αυτές αναφέρονται ακριβώς είτε παραφρασμένες. Επιπλέον, βεβαιώνω ότι αυτή η εργασία προετοιμάστηκε από εμένα προσωπικά, ειδικά ως διπλωματική εργασία, στο Τμήμα Μηχανικών Πληροφορικής και Ηλεκτρονικών Συστημάτων του ΔΙ.ΠΑ.Ε.

Η παρούσα εργασία αποτελεί πνευματική ιδιοκτησία τ... φοιτητ... ..... που την εκπόνησε/αν. Στο πλαίσιο της πολιτικής ανοικτής πρόσβασης, ο συγγραφέας/δημιουργός εκχωρεί στο Διεθνές Πανεπιστήμιο της Ελλάδος άδεια χρήσης του δικαιώματος αναπαραγωγής, δανεισμού, παρουσίασης στο κοινό και ψηφιακής διάχυσης της εργασίας διεθνώς, σε ηλεκτρονική μορφή και σε οποιοδήποτε μέσο, για διδακτικούς και ερευνητικούς σκοπούς, άνευ ανταλλάγματος. Η ανοικτή πρόσβαση στο πλήρες κείμενο της εργασίας, δεν σημαίνει καθ' οιονδήποτε τρόπο παραχώρηση δικαιωμάτων διανοητικής ιδιοκτησίας του συγγραφέα/δημιουργού, ούτε επιτρέπει την αναπαραγωγή, αναδημοσίευση, αντιγραφή, πώληση, εμπορική χρήση, διανομή, έκδοση, μεταφόρτωση (downloading), ανάρτηση (uploading), μετάφραση, τροποποίηση με οποιονδήποτε τρόπο, τμηματικά ή περιληπτικά της εργασίας, χωρίς τη ρητή προηγούμενη έγγραφη συναίνεση του συγγραφέα/δημιουργού.

Η έγκριση της διπλωματικής εργασίας από το Τμήμα Μηχανικών Πληροφορικής και Ηλεκτρονικών Συστημάτων του Διεθνούς Πανεπιστημίου της Ελλάδος, δεν υποδηλώνει απαραιτήτως και αποδοχή των απόψεων του συγγραφέα, εκ μέρους του Τμήματος.

«Αφιέρωση»

Πρόλογος

Σε αυτήν την ενότητα ο φοιτητής/φοιτήτρια μπορεί να αναφέρει τους λόγους που επέλεξε αυτήν την διπλωματική και το όφελος που είχε. Δεν θα πρέπει να ξεπεράσει τις 200 λέξεις.

Περίληψη

Σε αυτήν την ενότητα ο φοιτητής/φοιτήτρια θα πρέπει να περιγράψει συνοπτικά το θέμα και τα αποτελέσματα της διπλωματικής του εργασίας – δεν περιλαμβάνει βιβλιογραφικές αναφορές. Δεν θα πρέπει να ξεπεράσει τις 300 λέξεις.

«Τίτλος Διπλωματικής Εργασίας»

(στην αγγλική γλώσσα)

«Όνομα & Επώνυμο Φοιτητή»

(στην αγγλική γλώσσα)

Abstract

Σε αυτήν την ενότητα ο φοιτητής/ φοιτήτρια θα πρέπει να περιγράψει συνοπτικά το θέμα και τα αποτελέσματα της διπλωματικής του εργασίας στα αγγλικά – δεν περιλαμβάνει βιβλιογραφικές αναφορές. Δεν θα πρέπει να ξεπεράσει τις 300 λέξεις.

Ευχαριστίες

Σε αυτήν την ενότητα ο φοιτητής/ φοιτήτρια προαιρετικά μπορεί να ευχαριστήσει όσους αισθάνεται ότι συνέβαλαν (επιστημονικά, ηθικά, οικονομικά κτλ) στην ολοκλήρωση της διπλωματικής εργασίας.

Περιεχόμενα

Πρόλογος . . . iv
Περίληψη . . . v
Abstract . . . vi
Ευχαριστίες . . . vii
Περιεχόμενα . . . viii
Κατάλογος Σχημάτων . . . xi
Κατάλογος Πινάκων . . . xi
Συντομογραφίες . . . xii

1 Introduction **1**
1.1 Introduction . . . 1
1.2 Motivation . . . 1
1.3 Goal of the Thesis . . . 1
1.4 Contributions . . . 1
1.5 Honesty of the Data . . . 1
1.6 Structure of the Thesis . . . 1
1.7 Chapter Summary . . . 1

2 Background and Related Work **2**
2.1 Introduction . . . 2
2.2 Imitation Learning and Behaviour Cloning . . . 2
2.3 Interactive Imitation: DAgger . . . 2
2.4 Conditioned and Style-Aware Policies . . . 2
2.5 Player Modelling and Human-Like Game AI . . . 2
2.6 AI for Fighting Games . . . 2
2.7 Position of this Work . . . 2
2.8 Chapter Summary . . . 2

3 The Game and the Data Infrastructure **3**
3.1 Introduction . . . 3
3.2 The Game: DIENAMO . . . 3
3.2.1 Overview . . . 3
3.2.2 The Character Used in this Thesis: Skipler . . . 4
3.2.3 Scope . . . 5
3.2.4 Simulation Timing . . . 5
3.3 Event Telemetry . . . 5
3.3.1 Purpose and Format . . . 5
3.3.2 Corrections Made for this Thesis . . . 6
3.3.3 A Property of Human Telemetry . . . 6
3.4 Measuring Play Style: the Five Style Axes . . . 6
3.4.1 Definitions . . . 6
3.4.2 Choice of the Thresholds . . . 7
3.4.3 Interpretation . . . 7
3.4.4 The Same Measure Inside the Game . . . 8
3.5 Recording Demonstrations: the Imitation Logger . . . 8
3.5.1 Why a New Logger . . . 8
3.5.2 What is Recorded . . . 8
3.5.3 Observations . . . 8
3.5.4 Actions . . . 9
3.5.5 Intents versus Executed Actions . . . 9
3.5.6 Ordering of State and Action . . . 10
3.5.7 Known Limitation . . . 10
3.6 Automated Data Generation . . . 10
3.6.1 The Data-Collection Scene and the Director . . . 10
3.6.2 Match Loop . . . 10
3.6.3 Safeguards . . . 11
3.7 Fast and Faithful Collection: Parallel Locked-Step Execution . . . 11

3.7.1 The Problem . . . 11
3.7.2 Why Simply Accelerating Time Fails . . . 11
3.7.3 Locked Step . . . 11
3.7.4 Parallel Processes . . . 12
3.7.5 Validation . . . 12
3.7.6 Practical Consequence . . . 13
3.8 Chapter Summary . . . 13

4 Synthetic Teachers **14**
4.1 Introduction . . . 14
4.2 The Scripted Bot and its Personality Knobs . . . 14
4.3 From Knobs to Measured Style . . . 14
4.4 Persona Design and Coverage . . . 14
4.5 Why not RL Teachers . . . 14
4.6 Human-versus-Bot Diagnosis . . . 14
4.7 Bot v4 and Bot v5 . . . 14
4.8 Chapter Summary . . . 14

5 The Style-Conditioned Agent **15**
5.1 Introduction . . . 15
5.2 Problem Formulation . . . 15
5.3 Model . . . 15
5.4 Training . . . 15
5.5 Fine-tuning on Human Data . . . 15
5.6 In-Game Inference . . . 15
5.7 Online Adaptation: Mirror and Ghost . . . 15
5.8 Chapter Summary . . . 15

6 Experiments I: Synthetic Data **16**
6.1 Introduction . . . 16
6.2 Evaluation Protocol . . . 16
6.3 Style Fidelity from θ_3 to θ_6 . . . 16
6.4 Online Convergence . . . 16
6.5 Competence . . . 16
6.6 Chapter Summary . . . 16

7 Experiments II: Human Data **17**
7.1 Introduction . . . 17
7.2 Human Data Collection (P00) . . . 17
7.3 Fine-tuned Models θ_{6h} and θ_{6h2} . . . 17
7.4 Ghost and Mirror against the Author . . . 17
7.5 Blind Test . . . 17
7.6 Human-only Ablation (θ_H) . . . 17
7.7 Chapter Summary . . . 17

8 Experiments III: DAgger with a Scripted Expert **18**
8.1 Introduction . . . 18
8.2 Motivation . . . 18
8.3 Shadow Expert Design . . . 18
8.4 Three Variants . . . 18
8.5 Results . . . 18
8.6 Why it Fails: the Non-Markov Expert . . . 18
8.7 Chapter Summary . . . 18

9 Discussion and Limitations **19**
9.1 Introduction . . . 19
9.2 Synthetic Teachers as a Skill Ceiling . . . 19
9.3 One Character, One Stage . . . 19
9.4 Few Humans and a Single Expert Rater . . . 19
9.5 Metric Noise . . . 19
9.6 The REF Opponent . . . 19
9.7 Chapter Summary . . . 19

10 Conclusions and Future Work **20**
10.1 Introduction . . . 20
10.2 Conclusions . . . 20
10.3 Future Work . . . 20
10.4 Chapter Summary . . . 20

ΒΙΒΛΙΟΓΡΑΦΙΑ **21**

A Reproducibility: Scripts, Data Folders and Commands **23**

Κατάλογος Σχημάτων

3.1 Data pipeline. Each match produces two independent records: event telemetry, from which the style of each player is measured, and an imitation log of observations and actions. The two are joined by the match identifier when the dataset is built. 3
3.2 A Skipler vs Skipler match on Stage 1 of DIENAMO. The stage has a ground floor and three one-way platforms. The health bars of the two fighters are at the top of the screen, with the pause button between them. The fighter on the left, controlled by the imitation agent, is blocking. 4

Κατάλογος Πινάκων

3.1 The 67-value observation vector (per fighter, from its own point of view). 9
3.2 The nine action branches (20 values in total). 9
3.3 Validation of the parallel locked-step build against the original Editor collection (same 80 schedule entries). 12

Συντομογραφίες

Δ.Ε.	Διπλωματική Εργασία
ΔΙΠΑΕ	Διεθνές Πανεπιστήμιο Ελλάδος
Π.Ε.	Πτυχιακή Εργασία
BC	Behaviour Cloning
DAgger	Dataset Aggregation
EMA	Exponential Moving Average
IL	Imitation Learning
MLP	Multi-Layer Perceptron
NLL	Negative Log-Likelihood
RL	Reinforcement Learning
REF	Fixed reference scripted opponent used in evaluations

Κεφάλαιο 1ο: Introduction

1.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

1.2 Motivation

[TODO: Bots in games that play like people: scripted AI is predictable, RL agents are strong but non-human. Players want opponents/partners with a recognisable human style.]

1.3 Goal of the Thesis

[TODO: One general style-conditioned imitation agent for DIENAMO / Head of Hell that adapts online to the style of the player it faces (mirror) or of a stored profile (ghost), without retraining.]

1.4 Contributions

[TODO: (i) telemetry-based 5-axis style measure, validated C# = Python; (ii) parallel locked-step data generation, validated against the Editor; (iii) synthetic scripted teachers made more human from a measured human-vs-bot diagnosis (bot v3→v5); (iv) style-conditioned BC agent with online EMA style; (v) evaluation on synthetic and human data, incl. human fine-tune and a blind test; (vi) a negative, explained DAgger result with a scripted expert.]

1.5 Honesty of the Data

[TODO: State early: most training data are synthetic (scripted bots) and are always called synthetic; human data come from the author (P00) and are reported as such.]

1.6 Structure of the Thesis

[TODO: One paragraph per chapter.]

1.7 Chapter Summary

[TODO: recap of the main points (required by the template).]

Κεφάλαιο 2ο: Background and Related Work

2.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

2.2 Imitation Learning and Behaviour Cloning

[TODO: Supervised learning from demonstrations [1, 2]; compounding error / covariate shift.]

2.3 Interactive Imitation: DAgger

[TODO: [3]; assumption that the expert can label any visited state from the state alone; noise-injection alternatives [4]; causal confusion and history [5].]

2.4 Conditioned and Style-Aware Policies

[TODO: Conditional IL [6]; diverse behaviours / latent styles [7, 8]; GAIL [9].]

2.5 Player Modelling and Human-Like Game AI

[TODO: [10]; procedural personas [11]; human-like movement [12]; believability and Turing tests for bots [13, 14].]

2.6 AI for Fighting Games

[TODO: FightingICE and related work [15]; Unity ML-Agents [16].]

2.7 Position of this Work

[TODO: What is new here: telemetry style vector as conditioning signal, online adaptation by EMA only, synthetic teachers tuned to human statistics.]

2.8 Chapter Summary

[TODO: recap of the main points (required by the template).]

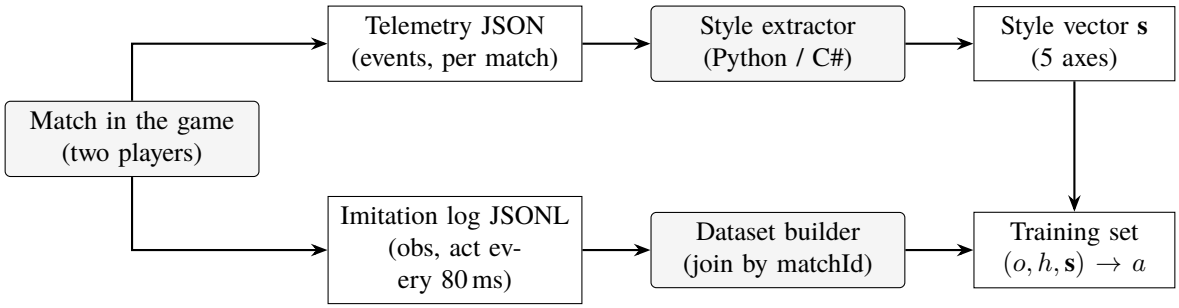
Κεφάλαιο 3ο: The Game and the Data Infrastructure

3.1 Introduction

Every result in this thesis rests on two measurements taken inside the game: *what a player did* (the button presses an imitation agent must learn to reproduce) and *how a player plays* (the style vector that conditions the agent and against which its fidelity is judged). If either measurement is wrong, the agent learns the wrong thing and the evaluation reports the wrong number. This chapter therefore describes the game and the infrastructure that was built around it before any model was trained.

Section 3.2 introduces the game and the scope chosen for the thesis. Section 3.3 describes the event telemetry and Section 3.4 the five style axes computed from it, together with the test that proves the in-game implementation equal to the offline one. Section 3.5 presents the imitation logger, which records observations and actions for training. Section 3.6 describes the automated data-generation mode, and Section 3.7 the parallel, locked-step execution that made large collections practical, including its validation against the original single-process collection.

Figure 3.1 summarises how the parts fit together.



Σχήμα 3.1: Data pipeline. Each match produces two independent records: event telemetry, from which the style of each player is measured, and an imitation log of observations and actions. The two are joined by the match identifier when the dataset is built.

3.2 The Game: DIENAMO

3.2.1 Overview

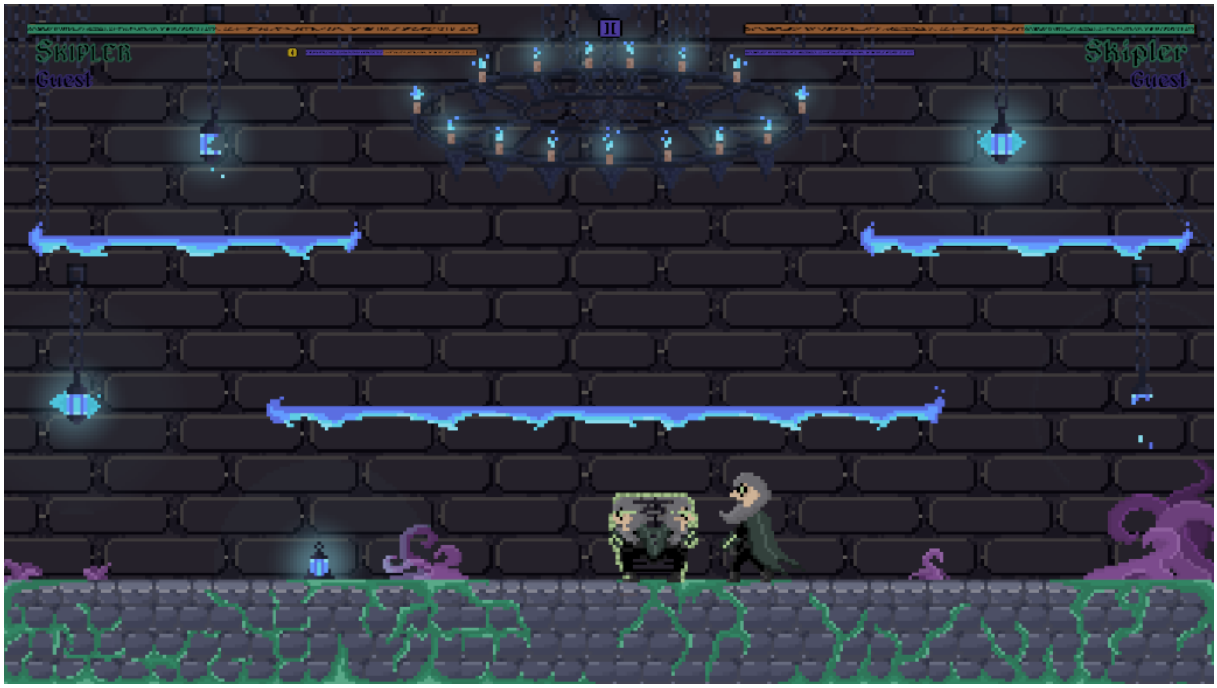
DIENAMO is a two-dimensional platform fighting game developed by the author in the Unity engine. Two fighters face each other on a stage made of a ground floor and one-way platforms that can be jumped onto from below and dropped through from above. A round ends when the health of one fighter, initially 100 points, reaches zero.

The game contains ten playable characters (Chiback, Fin, Lazy Bigus, Lithra, Lupen, Rager, Skipler, Steelager, Vander and Visvia). They share one control scheme and one set of core mechanics, while each character gives these mechanics its own form (its "kit"):

- **movement**: walking left and right, jumping, and dropping through a platform;

- **quick attack**: a fast, low-commitment attack;
- **heavy attack**: a slower, stronger attack with a wind-up before the hit; it can be cancelled during the wind-up, and cancelling (feinting) is a deliberate tactic;
- **special ability**: a character-specific move with a cooldown;
- **charge attack**: the fighter holds a button to charge, then releases a strong attack; a charge can be released, cancelled, or broken by the opponent;
- **block**: held; it reduces incoming damage and slows the fighter down, but does not stop movement;
- **parry**: a timed defensive press that, when successful, absorbs the attack and allows a counter.

Figure 3.2 shows a match as the player sees it.



Σχήμα 3.2: A Skipler vs Skipler match on Stage 1 of DIENAMO. The stage has a ground floor and three one-way platforms. The health bars of the two fighters are at the top of the screen, with the pause button between them. The fighter on the left, controlled by the imitation agent, is blocking.

3.2.2 The Character Used in this Thesis: Skipler

All experiments use a single character, Skipler, in mirror matches (Skipler against Skipler). Skipler's kit is mobility-oriented:

- the quick attack (*Blink*) is a short dash that damages the opponent on contact; its direction is the direction the player is holding, and with no direction held it goes to the right;
- the special ability is a longer damaging dash that leaves an after-image (a "double") behind, with a cooldown of 8 s;
- the heavy attack hits 0.567 s after it starts;

- a charge takes 0.5 s to complete, and the fighter cannot move while charging.

The fact that both of Skipler's main attacks are themselves movements matters later: it means that "attacking" and "moving" are intertwined for this character, and it explains why a correct choice of direction for the quick attack turned out to be one of the most important properties of a good synthetic teacher (Chapter 4).

3.2.3 Scope

The following restrictions were decided at the start of the work and hold for all experiments:

- **one character, mirror matches**: Skipler against Skipler. This keeps the observation space identical to the one the agent was trained on, makes the measured styles of different players directly comparable, and gives a clean claim. The cost (no generalisation to other characters or matchups) is discussed in Chapter 9; applying the same recipe to other characters is future work (Chapter 10);
- **one stage** (Stage 1) and the **Classic** ruleset: 100 health points, standard speed and abilities, no portals and no power-ups. Portals and power-ups belong to a separate custom game mode and are out of scope;
- **one round per match** in all automated collections, so that every match is one independent sample of play;
- **one-versus-one** only.

3.2.4 Simulation Timing

The game logic runs on Unity's fixed physics step of 0.02 s, i.e. 50 Hz [17]; rendering, and the per-frame decisions of the scripted bots, run once per rendered frame. Both facts become important in Section 3.7, where the speed of data generation is increased without changing what the game simulates.

3.3 Event Telemetry

3.3.1 Purpose and Format

The game records every completed round as a JSON document (telemetry schema version 2.0). A document contains match metadata — a unique match identifier, the characters, the profile or synthetic persona of each side, the winner, the fight duration — and a time-stamped list of events for each player. Events are either *actions* (e.g. `Jump`, `Quick`, `HeavyStart`, `ParryAttempt`, `BlockStart`/`BlockEnd`) or *outcomes* (e.g. `HitAttempt`, `Miss`, `DamageApplied` with flags for parried and critical hits).

Telemetry is written only for rounds that reached the start of the fight (after the countdown) and finished normally; a round quit in the middle writes nothing. Telemetry produced by automated collections is written to separate folders, so that synthetic matches are never mixed with the profiles of real players.

3.3.2 Corrections Made for this Thesis

The telemetry existed before this thesis, but an audit showed that several actions were logged inconsistently, per character, or not at all. Because the style vector is computed from these events, the telemetry was revised before any data were collected. The main rules adopted were:

- **one central rule per action**: each action is logged in one place in the shared character code, instead of separately in each character's script;
- **every change of walking direction is logged** (an earlier rate limit hid short movements);
- **the decision, not only the outcome, is logged**: a `HeavyStart` event marks every heavy attack that begins, including those cancelled before the hit;
- **charge, parry and multi-hit damage are logged centrally**: charge release, cancel and break are derived from the change of state around the charge check; a parried attack is recorded on the defender's damage event, and the remaining hits of the same parried move are tagged so that they are not counted as dodges; multi-hit moves produce a single aggregated damage event;
- **marker events do not affect style**: new events that exist only to mark a decision time or a state change (e.g. `ChargeStart`, `BlockStart`) are excluded from the style counts, so that adding them did not move any style axis.

Each change was verified with test matches covering the affected characters (e.g. forms, combos, parry on the first and on the second hit of a multi-hit move, a quit in the middle of a round). These corrections concern all ten characters; for this thesis only Skipler's events are used.

3.3.3 A Property of Human Telemetry

One property of the telemetry must be kept in mind whenever human and synthetic players are compared. When a human player *holds* a key or presses it repeatedly within a very short time, the telemetry can record the same action several times in consecutive frames (for example repeated `DropPlatform` events). In the first human-versus-bot comparison, 13% of all transitions between consecutive human actions were `DropPlatform`→`DropPlatform` within one frame. Scripted bots do not produce this pattern. Any comparison of action *rhythm* between humans and bots therefore merges such repeats first (Chapter 4). The style axes below are not affected, because they count actions by category over the whole round or are defined on time.

3.4 Measuring Play Style: the Five Style Axes

3.4.1 Definitions

The style of a player in one round is summarised by a vector of five numbers,

$$\mathbf{s} = \left(r_{\text{att}},\ r_{\text{mob}},\ r_{\text{def}},\ \tau_{\text{mob}},\ \tau_{\text{blk}}\right), \tag{3.1}$$

computed from the round's telemetry. The first three are *action shares*. Every counted action of the player is assigned to one of three categories:

- **attack**: quick attack, heavy attack (counted when it starts, so feints count), special ability, charge release;
- **mobility**: jump, drop through a platform, dash, and each *walking segment* of at least 0.2 s;
- **defence**: each *block segment* of at least 0.1 s, and each parry attempt.

With N_{att}, N_{mob}, N_{def} the counts and $N = N_{\text{att}} + N_{\text{mob}} + N_{\text{def}}$,

$$r_{\text{att}} = \frac{N_{\text{att}}}{N}, \qquad r_{\text{mob}} = \frac{N_{\text{mob}}}{N}, \qquad r_{\text{def}} = \frac{N_{\text{def}}}{N}. \tag{3.2}$$

The other two axes are *time ratios*. With T the fight duration of the round, T_{walk} the total time spent walking and T_{block} the total time spent blocking,

$$\tau_{\text{mob}} = \min\left(1, \frac{T_{\text{walk}}}{T}\right), \qquad \tau_{\text{blk}} = \min\left(1, \frac{T_{\text{block}}}{T}\right). \tag{3.3}$$

Rounds with fewer than 10 counted actions or shorter than 5 s are discarded, because their shares are too noisy to describe a style. In the code the five axes are named `attack_rate`, `mobility_rate`, `defense_rate`, `mobility_time_ratio` and `block_time_ratio`.

3.4.2 Choice of the Thresholds

The two segment thresholds were chosen from duration histograms rather than set arbitrarily. For walking, segments shorter than 0.2 s were about one third of all segments but only about 7% of the time spent moving: they are taps, jitter and animation cancels rather than decisions to move. Blocks are different: a block protects immediately, so short blocks are real defensive actions. In a histogram of 51 bot blocks only 3 were shorter than 0.1 s, and all 3 were cancelled within one or two frames by another action. Blocks therefore use the lower threshold of 0.1 s. Repeated short blocks are *not* merged into one, because rapid blocking is a legitimate defensive tactic; their frequency is only monitored.

3.4.3 Interpretation

The three shares sum to one, so they describe *how a player distributes their actions*, not how aggressive a player feels or looks. This distinction matters when the numbers are read. A player who plays aggressively by constantly closing the distance produces many walking segments and jumps and is measured as mostly "mobile". The author, who plays aggressively, measures about $r_{\text{att}} = 0.26$ and $r_{\text{mob}} = 0.71$. Similarly, because blocking does not stop movement (the fighter walks more slowly while blocking), a high defence share lowers the mobility share only through the accounting of Equation (3.2), not because the player stands still. The two time ratios complement the shares with information the shares cannot carry, namely how *long* the player moves and blocks.

The style of a player is, finally, also influenced by the opponent: the same player measures differently

against different opponents. For this reason all style-fidelity measurements in this thesis are made against one fixed reference opponent (REF, Chapter 6).

3.4.4 The Same Measure Inside the Game

The offline extractor is written in Python. The online part of the system (Chapter 5) must measure the style of a human *during* a session, inside the game, after every round. The extractor was therefore ported to C# (`StyleAxes.cs`). An equivalence test ran both implementations on 931 telemetry files containing 1,770 player-rounds: the difference between the two was zero on every round. A second script verifies, after each round of an online session, that the value used in the game matches the Python value for that round.

3.5 Recording Demonstrations: the Imitation Logger

3.5.1 Why a New Logger

The game already contained an imitation logger, but an audit showed it was not usable for this thesis: it read the physical keyboard directly (so it could not record bots), it recorded once per rendered frame (so the sampling rate depended on the computer), it had no match identifier (so it could not be joined with telemetry), and the training script split the data by random frames, which leaks almost identical neighbouring frames between the training and test sets. A new logger was written.

3.5.2 What is Recorded

The logger records *both* players of every match, each as its own stream of samples. It samples every 4 physics steps, i.e. every 80 ms (12.5 samples per second of game time); the agent later decides at exactly the same rate. Instead of reading the keyboard, it reads the character's input provider, an interface implemented by the keyboard, the scripted bot and the agent alike. The same logger therefore records humans and bots in the same way. Each sample contains the match identifier, the side, the kind of player (human, scripted, agent) and its persona or model name, the tick and time, the observation vector and the action vector. One file is written per match and is named by the same match identifier as the telemetry document. The style vector is *not* stored in the samples; it is joined later from the telemetry of the same match (Figure 3.1). The logger runs only in the dedicated data-collection scene; in the normal game only telemetry is recorded.

3.5.3 Observations

The observation is the 67-value vector already used by the game's reinforcement-learning agents, so that all agents share one interface (Table 3.1). It describes the relative position and spacing of the two fighters, their velocities and health, which character each one plays, the state flags of both fighters, and facing information. Positions and velocities are scaled and clipped to $[-1, 1]$. The agent of Chapter 5 transforms this vector into an egocentric frame and adds history and a quick-ready feature; those additions are computed from the logged data and are described there.

Πίνακας 3.1: The 67-value observation vector (per fighter, from its own point of view).

Group	Values	Content
Relative position	2	opponent position minus own position, x and y, scaled
Spacing	2	absolute horizontal and vertical distance, scaled
Velocities	4	own and opponent velocity, x and y
Health	2	own and opponent health / 100
Own character	10	one-hot over the ten characters
Opponent character	10	one-hot over the ten characters
Own state	14	grounded, blocking, casting, stunned, knocked, charging, charged, ability on cooldown, cooldown fraction, can cast, can parry, quick / heavy attack active, parrying
Own disabled flags	6	quick, heavy, block, special, charge, jump disabled
Opponent state	14	the same 14 state values for the opponent
Facing	3	direction to the opponent, own facing, facing the opponent (yes/no)
Total	67	

3.5.4 Actions

The action is a vector of nine discrete branches, again identical to the interface of the game's reinforcement-learning agents (Table 3.2). Two kinds of buttons are distinguished. For *one-shot* buttons (jump, drop, quick, heavy, special, parry) the logged value means "pressed at least once since the previous sample". For *held* buttons (move, block) the logged value is the state at the sampling tick. The charge branch combines the two rules: it is 0 when the charge button is not held, 1 when it is held, and 2 when it was released since the previous sample (a release has priority, because releasing is what fires the charge attack). Without the one-shot rule, a press shorter than 80 ms that fell between two samples would be lost.

Πίνακας 3.2: The nine action branches (20 values in total).

Branch	Values	Meaning
Move	3	none / left / right (held)
Jump	2	pressed since last sample
Drop through platform	2	pressed since last sample
Quick attack	2	pressed since last sample
Heavy attack	2	pressed since last sample
Block	2	held
Special ability	2	pressed since last sample
Charge	3	not held / held / released since last sample
Parry	2	pressed since last sample

3.5.5 Intents versus Executed Actions

The logger records *intents*: what the player pressed. Telemetry records *executed* actions. The two can differ, for example when a quick attack is pressed while it is still on cooldown. This is the desired behaviour: the agent must learn which buttons to press in a given situation, exactly as a human does, while the style measure describes what actually happened in the round.

3.5.6 Ordering of State and Action

One error in the first version of the dataset must be reported because it changed the results. In dataset v1 the observation of a tick was recorded *after* the presses of that same tick had been applied, so the input to the model already reflected part of the action it was meant to predict. The error was found after the first two models (θ_0, θ_1) and was corrected from dataset v2 onwards: every sample now pairs the state *before* the decision with the decision taken in it. All models reported in Chapters 6–8 were trained on corrected data.

3.5.7 Known Limitation

Presses made in the last 80 ms before a round ends are not recorded, because no further sample is taken. This affects at most one sample per player per round.

3.6 Automated Data Generation

3.6.1 The Data-Collection Scene and the Director

Synthetic data are produced by matches between scripted bots, and evaluations by matches between agents and bots, all without a human at the keyboard. A dedicated scene (`ILDataScene`) runs the normal fight logic — countdown, fight start, round end, telemetry and logger — without menus, controlled by a component called the *director*. The director reads a *schedule*: a list of match-ups, each giving for both sides the type of player and its parameters, and the number of matches to play. A side can be

- a **scripted persona**: the game's scripted bot with fixed personality knobs and a skill level (Chapter 4);
- an **imitation agent**: a trained model with a style source, either a fixed style vector or the online style of another side (`mirrorOf`, Chapter 5).

Later experiments added further optional fields: a different model per agent, a "shadow" expert for DAgger (Chapter 8) and an in-game rating window for the blind test (Chapter 7).

3.6.2 Match Loop

For every entry the director applies the scope of Section 3.2 (Skipler, one round, power-ups disabled; in the built game it also pins Stage 1 and the Classic ruleset, because the build keeps its own saved settings), redirects telemetry and logs into the collection's own folder, and starts the match. After a match is saved it records it in a run log and reloads the scene for the next one. The run log makes collections resumable: a collection interrupted for any reason continues from the first unsaved match. A match with no result after 180 s is treated as a stalemate, restarted and not counted; a match-up that stalls twice is skipped. When the schedule is finished the director stops.

3.6.3 Safeguards

Two mistakes made during the work led to permanent checks, because both would have silently corrupted data:

- **unsaved scene changes**: a change made in the Unity Inspector without saving the scene applies only to the first match; after the first reload the scene file is read again. In one human session this caused 24 rounds to be recorded under the wrong style label. They were corrected through a separate relabelling file, without editing the match files (Chapter 7). Since then, if the schedule, the output folder or the model differ after a reload, the director stops the collection with a message;
- **recorded conditions**: every run writes an environment file with all conditions that could affect the data (stage, ruleset, time settings, frame rate, graphics), so that any collection can be checked afterwards.

3.7 Fast and Faithful Collection: Parallel Locked-Step Execution

3.7.1 The Problem

A collection in the Unity Editor runs in real time: one match of about 35 s of fight plus its countdown takes about 40 s, so the 520 matches of a full dataset take hours. Every change to the synthetic teachers (Chapter 4) requires a new collection, which made the cost of each iteration prohibitive.

3.7.2 Why Simply Accelerating Time Fails

The obvious solution, running the game faster by setting Unity's time scale to 3, was tested in a pilot and rejected. At time scale 3 each rendered frame advances the game by $3/60$ s instead of $1/60$ s, so the game is simulated at 20 frames per second of game time instead of 60. The scripted bots decide once per frame and therefore made a third of their usual decisions. The effect on the data was large: the walking-time axis of one defensive persona shifted by $d = 1.5$ (Cohen's d), and the win rate of one mobile persona rose from 53% to 87%. Accelerated time changes the behaviour it is supposed to record.

3.7.3 Locked Step

The solution separates *game time* from *real time*. In the built game the director sets Unity's capture frame time to exactly $1/60$ s with time scale 1 and an unlimited frame rate [18]. Each rendered frame then advances the game by exactly $1/60$ s, whatever the real time it took to compute. The game sees exactly the same sequence of frames as a real-time match — every frame is simulated, and every bot decision is taken — but it no longer waits between frames. In the words used in the defence: accelerating time is like tearing out two of every three pages of a book, while locked step reads every page, only faster. Locked step is even more stable than real time, where a slow computer produces longer frames.

A code audit confirmed that no gameplay code (characters, scripted bot, agent) uses real time. Real time appears only in file names and timestamps, in the pause between matches, in a re-find timer of the logger, and in a safety timer of reinforcement-learning training, which does not run in this scene.

3.7.4 Parallel Processes

The Editor can run only one game at a time, so parallelism requires several copies of the *built* game. A command-line switch makes the build start directly in the data-collection scene: sound is muted, the game keeps running in the background, leftovers of the main menu are removed (so that matches carry no player identity, as in the Editor), and only the visual elements of the stage are hidden — colliders, animations and scripts are untouched. Without the switch the build is the normal game. A launcher script starts N processes, each with one *shard* of the schedule and its own output folder and run log, and merges the shards into one ordinary collection folder at the end. Shards can be formed by index or by pairs; the pair mode keeps all matches of a pair of personas in one process, which is required when an agent follows the online style of its opponent across consecutive matches.

3.7.5 Validation

Faster data are only useful if they are the same data. The first 80 entries of the main synthetic schedule were therefore collected again with the parallel build (4 processes) and compared match by match with the original Editor collection (Table 3.3).

Πίνακας 3.3: Validation of the parallel locked-step build against the original Editor collection (same 80 schedule entries).

Measure	Editor, real time	Parallel build, 4 processes
Wall-clock time for 80 matches	≈50 min	2.2 min
Simulated frames per game second	59.99	60.00
Logger samples per game second	12.5	12.5
Hit rate	0.384	0.394
Damage per second	2.63	2.57
Fight duration (s)	34.0	34.5
Style difference per persona, mean z^2	1.10 (Editor vs Editor)	1.15
Paired t per match, all measures (160 pairs)	all $\lvert t\rvert < 1.5$	

The style comparison needs a baseline. The mean z^2 of the per-persona style difference measures how far two collections differ relative to the match-to-match noise of each persona; a value near 1 means “no more different than chance”. Two halves of the *Editor* collection differ by 1.10; the parallel build differs from the Editor by 1.15. In the paired comparison (same schedule entry and same side, 160 pairs) no measure showed a significant difference: attack $t = -1.04$, mobility $+1.41$, defence -0.60, walking time $+0.14$, block time -0.27, hit rate $+0.41$, damage per second -0.61, duration -0.03. The data of the parallel build are therefore equivalent to those of the Editor at real time, while being collected about 24 times faster. The speed-up scaled almost linearly with the number of processes (4.2 min with 2 processes, 2.2 min with 4), limited by the number of processor cores.

The validation also revealed one defect: the first match of every process carried a guest player identity, because a leftover menu object was destroyed only at the end of the first frame. It had no effect on the data, which identify synthetic players by persona, and it was fixed by destroying the object immediately; a check collection confirmed 80 out of 80 matches without identity.

3.7.6 Practical Consequence

With the parallel build a complete synthetic dataset of 520 matches is collected in about 15 minutes instead of several hours. This changed the method of the thesis: the synthetic teachers could be diagnosed, changed and re-collected several times (bot v3 $\rightarrow$ v4 $\rightarrow$ v5, Chapter 4), and every evaluation in Chapters 6–8 could use enough matches to estimate its own noise.

3.8 Chapter Summary

This chapter described the game, the scope chosen for the thesis and the infrastructure that produces its data. The game is a two-dimensional platform fighter with ten characters; all experiments use Skipler in mirror matches on one stage with the Classic ruleset. The event telemetry was revised so that every action is logged once and consistently, and the style of a player is measured from it as five numbers: three action shares and two time ratios, with thresholds chosen from data. The in-game C# implementation of this measure is identical to the Python one on 1,770 rounds. A new imitation logger records the observations and actions of both players every 80 ms through the same interface for humans and bots, and is joined with the telemetry by match identifier. An automated director plays scheduled matches without menus, with safeguards against two silent failure modes. Finally, a parallel locked-step build collects the same data as the real-time Editor — shown by a match-by-match comparison — about 24 times faster, which made it possible to iterate on the synthetic teachers described in the next chapter.

Κεφάλαιο 4ο: Synthetic Teachers

4.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

4.2 The Scripted Bot and its Personality Knobs

[TODO: SimpleBotController; knobs aggression / defence / mobility; PersonaMul $= 4^{(k-0.5)\cdot 2}$; pilots 1–2 (log §5).]

4.3 From Knobs to Measured Style

[TODO: Labels are the *measured* style, not the knobs; knobs entangle style and skill.]

4.4 Persona Design and Coverage

[TODO: 30 personas, Latin hypercube [19], hold-outs; empty corner (low D, high M) and the extension X01–X10 (log §7, §15).]

4.5 Why not RL Teachers

[TODO: RL pilot: 97% wins, non-human (log §11).]

4.6 Human-versus-Bot Diagnosis

[TODO: 15 metrics, Cohen's d [20]; measurement artefact of held keys (log §26).]

4.7 Bot v4 and Bot v5

[TODO: Changes A, B, D, E, F (C disabled); G, H, I; tables of human metrics (log §26, §30).]

4.8 Chapter Summary

[TODO: recap of the main points (required by the template).]

Κεφάλαιο 5ο: The Style-Conditioned Agent

5.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

5.2 Problem Formulation

[TODO: Policy $\pi_\theta(a \mid o, h, s)$; observation, history, style vector.]

5.3 Model

[TODO: MLP; joint action vocabulary (118/125 combinations); egocentric frame; quick-ready flag; history features (log §8).]

5.4 Training

[TODO: Split by match 70/15/15 + unseen personas; NLL model selection; sampling at play time; bias calibration.]

5.5 Fine-tuning on Human Data

[TODO: finetune_bc_joint: start from θ_6, 50% bot replay, early stopping on human validation (log §31, §36).]

5.6 In-Game Inference

[TODO: ILAgentController, decision every 80 ms, Python/Unity self-test (log §9).]

5.7 Online Adaptation: Mirror and Ghost

[TODO: EMA $\alpha = 0.35$, neutral start, clamp to training range (log §12).]

5.8 Chapter Summary

[TODO: recap of the main points (required by the template).]

Κεφάλαιο 6ο: Experiments I: Synthetic Data

6.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

6.2 Evaluation Protocol

[TODO: Fixed opponent REF; style error / noise; correlation; identification $x/16$ and why it is noisy (log §17, §35).]

6.3 Style Fidelity from θ_3 to θ_6

[TODO: Table + figure teacher vs agent per axis.]

6.4 Online Convergence

[TODO: Curves $\theta_4/\theta_5/\theta_6/\theta_{6h}$: 2.28 $\rightarrow$ 1.18 $\rightarrow \approx$1.0.]

6.5 Competence

[TODO: Wins, hit rate, damage, heavy cancels vs teacher.]

6.6 Chapter Summary

[TODO: recap of the main points (required by the template).]

Κεφάλαιο 7ο: Experiments II: Human Data

7.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

7.2 Human Data Collection (P00)

[TODO: Natural vs instructed styles; relabelling incident; only P00N is the human reference (log §32).]

7.3 Fine-tuned Models θ_{6h} and θ_{6h2}

[TODO: Offline NLL on unseen human rounds 2.92 $\rightarrow$ 1.09 $\rightarrow$ 1.00 (log §36).]

7.4 Ghost and Mirror against the Author

[TODO: Behaviour distance; play profile figure; mirror results.]

7.5 Blind Test

[TODO: In-game rating window; 16 rounds; single expert rater, descriptive only (log §37).]

7.6 Human-only Ablation (θ_H)

[TODO: [TODO: pending experiment: do the scripted teachers help?]]

7.7 Chapter Summary

[TODO: recap of the main points (required by the template).]

Κεφάλαιο 8ο: Experiments III: DAgger with a Scripted Expert

8.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

8.2 Motivation

[TODO: Agent-only errors: heavy cancel 23% vs 12%, passivity.]

8.3 Shadow Expert Design

[TODO: Bot runs in the shadow on the agent's character, never controls it; expertAct logged; own-action history (log §34).]

8.4 Three Variants

[TODO: θ_7 full labels, θ_{7a} press-only labels, θ_{7b} synchronised expert.]

8.5 Results

[TODO: Offline NLL -36% but flicker in play; θ_{7a} heavy cancel $22.7\% \rightarrow 19.0\%$.]

8.6 Why it Fails: the Non-Markov Expert

[TODO: Hidden timers / commitments violate the DAgger assumption; offline NLL did not predict play.]

8.7 Chapter Summary

[TODO: recap of the main points (required by the template).]

Κεφάλαιο 9ο: Discussion and Limitations

9.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

9.2 Synthetic Teachers as a Skill Ceiling

[TODO: write]

9.3 One Character, One Stage

[TODO: write]

9.4 Few Humans and a Single Expert Rater

[TODO: write]

9.5 Metric Noise

[TODO: Identification $x/16$; per-player convergence values do not replicate.]

9.6 The REF Opponent

[TODO: Does not press enough to require block/parry timing.]

9.7 Chapter Summary

[TODO: recap of the main points (required by the template).]

Κεφάλαιο 10ο: Conclusions and Future Work

10.1 Introduction

[TODO: chapter introduction: what this chapter covers.]

10.2 Conclusions

[TODO: write]

10.3 Future Work

[TODO: Other characters with the same recipe; cross matchups; more humans; fine-tune per session (online (b)).]

10.4 Chapter Summary

[TODO: recap of the main points (required by the template).]

ΒΙΒΛΙΟΓΡΑΦΙΑ

[1] D. A. Pomerleau, "ALVINN: An autonomous land vehicle in a neural network," in *Advances in Neural Information Processing Systems 1*, pp. 305–313, 1989.

[2] A. Hussein, M. M. Gaber, E. Elyan, and C. Jayne, "Imitation learning: A survey of learning methods," *ACM Computing Surveys*, vol. 50, no. 2, pp. 21:1–21:35, 2017.

[3] S. Ross, G. J. Gordon, and J. A. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the 14th International Conference on Artificial Intelligence and Statistics (AISTATS)*, vol. 15 of *PMLR*, pp. 627–635, 2011.

[4] M. Laskey, J. Lee, R. Fox, A. Dragan, and K. Goldberg, "DART: Noise injection for robust imitation learning," in *Proceedings of the 1st Conference on Robot Learning (CoRL)*, 2017.

[5] P. de Haan, D. Jayaraman, and S. Levine, "Causal confusion in imitation learning," in *Advances in Neural Information Processing Systems 32*, 2019.

[6] F. Codevilla, M. Müller, A. López, V. Koltun, and A. Dosovitskiy, "End-to-end driving via conditional imitation learning," in *IEEE International Conference on Robotics and Automation (ICRA)*, 2018.

[7] Z. Wang, J. Merel, S. Reed, G. Wayne, N. de Freitas, and N. Heess, "Robust imitation of diverse behaviors," in *Advances in Neural Information Processing Systems 30*, 2017.

[8] Y. Li, J. Song, and S. Ermon, "InfoGAIL: Interpretable imitation learning from visual demonstrations," in *Advances in Neural Information Processing Systems 30*, 2017.

[9] J. Ho and S. Ermon, "Generative adversarial imitation learning," in *Advances in Neural Information Processing Systems 29*, 2016.

[10] G. N. Yannakakis and J. Togelius, *Artificial Intelligence and Games*. Springer, 2018.

[11] C. Holmgård, A. Liapis, J. Togelius, and G. N. Yannakakis, "Evolving personas for player decision modeling," in *IEEE Conference on Computational Intelligence and Games (CIG)*, 2014.

[12] C. Thurau, C. Bauckhage, and G. Sagerer, "Learning human-like movement behavior for computer games," in *Proceedings of the 8th International Conference on the Simulation of Adaptive Behavior (SAB)*, 2004.

[13] P. Hingston, "A Turing test for computer game bots," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 1, no. 3, pp. 169–186, 2009.

[14] B. Soni and P. Hingston, "Bots trained to play like a human are more fun," in *IEEE International Joint Conference on Neural Networks (IJCNN)*, 2008.

[15] F. Lu, K. Yamamoto, L. H. Nomura, S. Mizuno, Y. Lee, and R. Thawonmas, "Fighting game artificial intelligence competition platform," in *IEEE 2nd Global Conference on Consumer Electronics (GCCE)*, 2013.

[16] A. Juliani, V.-P. Berges, E. Teng, A. Cohen, J. Harper, C. Elion, C. Goy, Y. Gao, H. Henry, M. Mattar, and D. Lange, "Unity: A general platform for intelligent agents," 2018. arXiv:1809.02627.

[17] Unity Technologies, "Time.fixeddeltatime – Unity scripting API." `https://docs.unity3d.com/ScriptReference/Time-fixedDeltaTime.html`, 2024. VERIFY the page and add the access date before submission.

[18] Unity Technologies, "Time.capturedeltatime – Unity scripting API." `https://docs.unity3d.com/ScriptReference/Time-captureDeltaTime.html`, 2024. VERIFY the page and add the access date before submission.

[19] M. D. McKay, R. J. Beckman, and W. J. Conover, "A comparison of three methods for selecting values of input variables in the analysis of output from a computer code," *Technometrics*, vol. 21, no. 2, pp. 239–245, 1979.

[20] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*. Lawrence Erlbaum Associates, 2nd ed., 1988.

ΠΑΡΑΡΤΗΜΑ A: Reproducibility: Scripts, Data Folders and Commands

[TODO: list of scripts (log §22, §39), data folders, how to rerun each experiment.]